

University
Politehnica
of Bucharest





Computer
Science and
Engineering
Department

MongoDB (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB



Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB



MongoDB – Replication

- mongod is the instance of a MongoDB server
- A replica set in MongoDB is a group of mongod processes that maintain the same data set.
- Replica sets provide
  - Redundancy
  - High availability
- Replica sets are the basis for all production deployments



MongoDB – Replication

- Replication provides
  - Redundancy
    - With multiple copies of data on different database servers, replication provides a level of fault tolerance against the loss of a single database server
  - High availability:
    - In some cases, replication can provide increased read capacity as clients can send read operations to different servers



MongoDB – Replication

- Maintaining copies of data in different data centers can increase
  - Data locality
  - Availability for distributed applications
- Additional copies can be maintained for dedicated purposes, such as:
  - Disaster recovery
  - Reporting
  - Backup



MongoDB – Replication

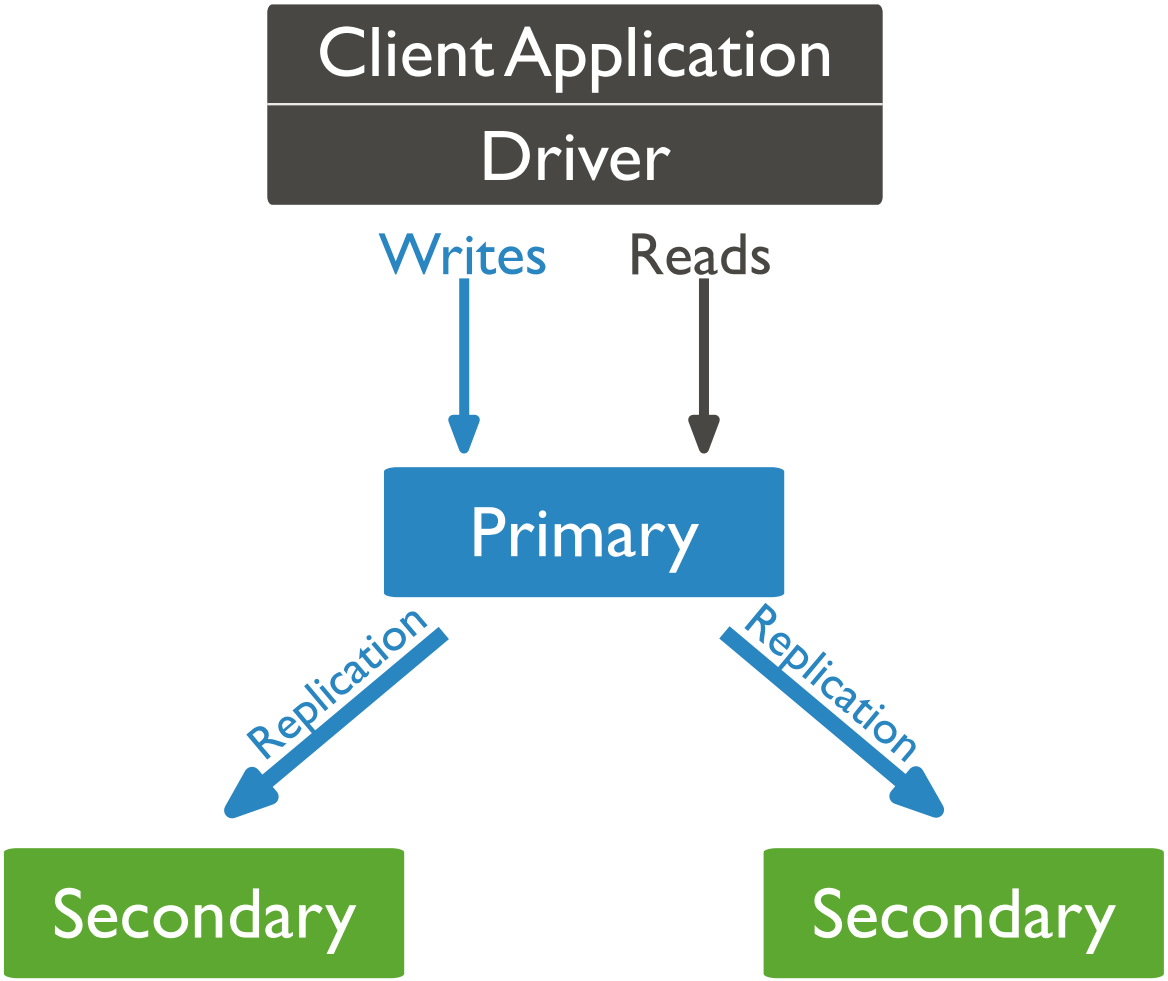
- A replica set is a group of mongod instances that maintain the same data set

- A replica set contains
  - Several data bearing nodes
  - Optionally one arbiter node

- The data bearing nodes
  - One and only one member is deemed the primary node
  - The other nodes are deemed secondary nodes

- The primary node
  - Receives all write operations
  - Records all changes to its data sets in its operation log

MongoDB – Replication

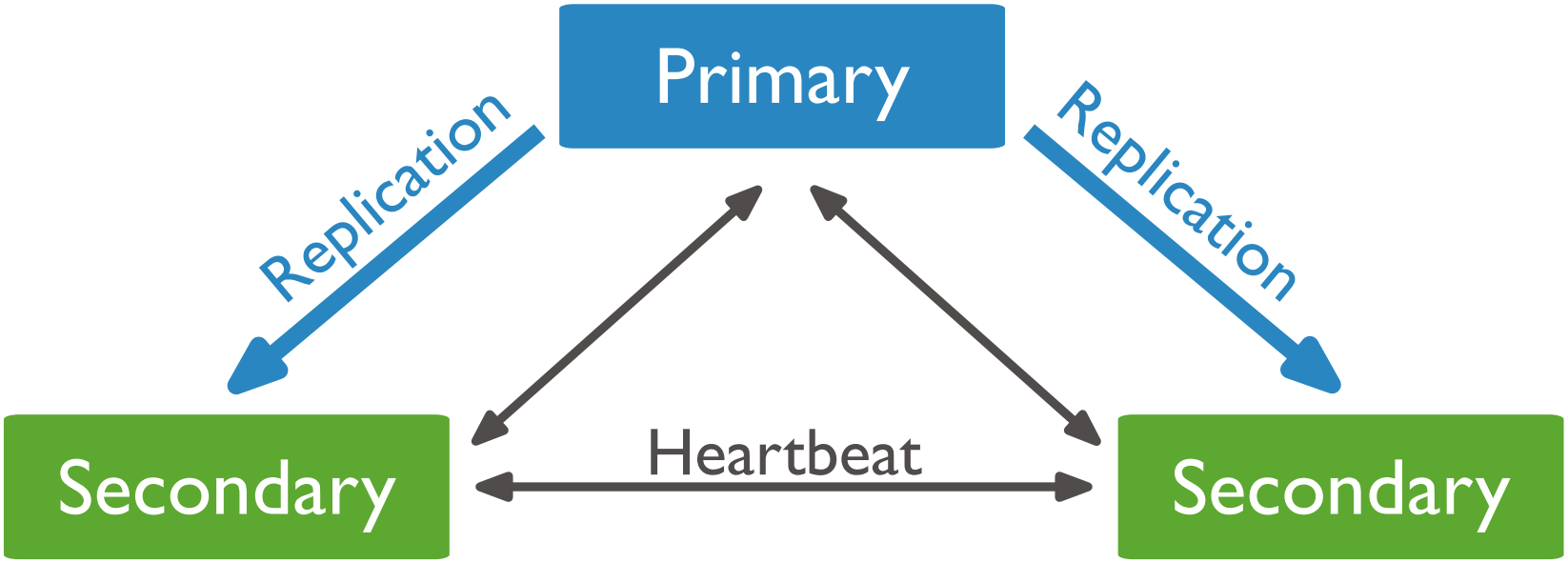




MongoDB – Replication

- The secondaries
  - Replicate the primary's oplog (operations log)
    - oplog: A special capped collection that keeps a rolling record of all operations that modify the data stored in the databases
    - capped collection: A fixed-sized collection that automatically overwrites its oldest entries when it reaches its maximum size
  - Apply the operations to their data sets such that the secondaries' data sets reflect the primary's data set
- If the primary is unavailable, an eligible secondary will hold an election to elect itself the new primary

MongoDB – Replication



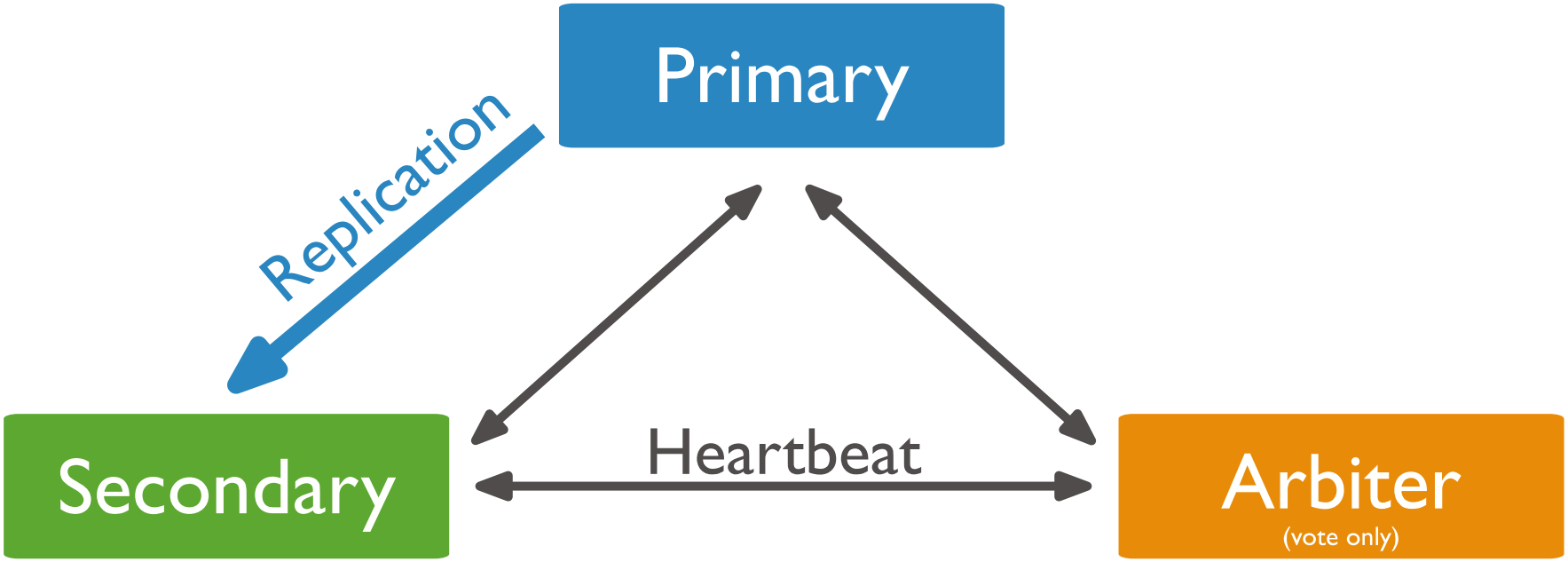


MongoDB – Replication

- To maintain the quorum of votes in a replica set an Arbiter can be added

- Arbiters
  - Do not maintain a data set
  - Respond to heartbeat and election requests by other replica set members
  - Can be a good way to provide replica set quorum functionality with a cheaper resource cost than a fully functional replica set member with a data set

MongoDB – Replication



MongoDB – Replication





MongoDB – Replication

- If your replica set has an even number of members, add an arbiter to obtain most votes in an election for primary
- Arbiters do not require dedicated hardware
- An arbiter will always be an arbiter whereas
  - A primary may step down and become a secondary during an election
  - A secondary may become the primary during an election



MongoDB – Replication

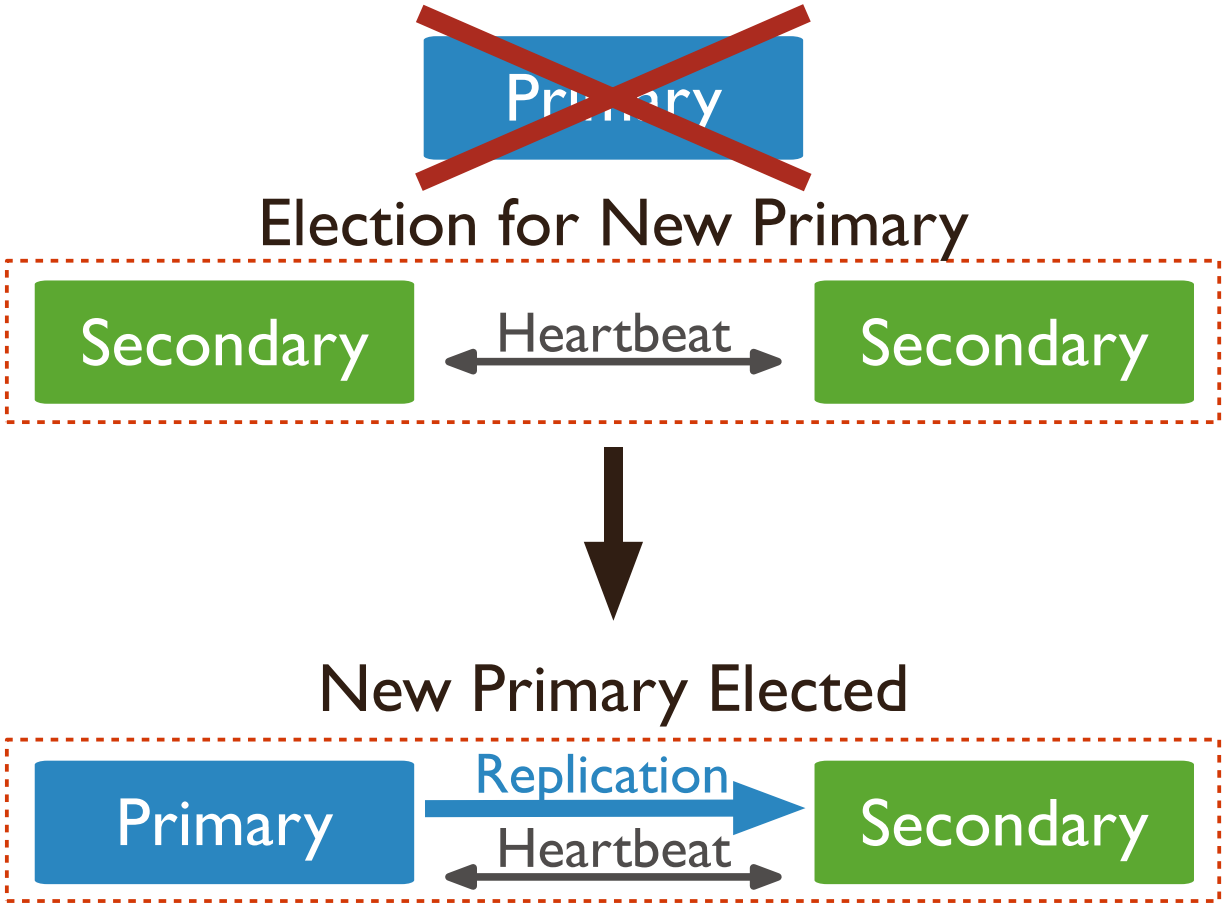
- Asynchronous Replication
  - Secondaries are updated by applying operations from the primary asynchronously
  - By applying operations after the primary, sets can continue to function despite the failure of one or more members



MongoDB – Replication

- Automatic Failover
  - If the primary node does not respond to a heartbeat for more than 10 seconds an eligible secondary holds an election to elect itself the new primary
  - The first secondary to hold an election and receive most of the members' votes becomes primary

MongoDB – Replication





MongoDB – Replication

- Replication Read Operations
  - By default, clients read from the primary
  - However, clients can specify to read from Secondaries
  - Because of asynchronous replication
    - There is no guaranty that secondaries have the latest information
    - The latest information is only on the primary node
  - Clients can see the results of writes before the write becomes durable



Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB



MongoDB – Sharding

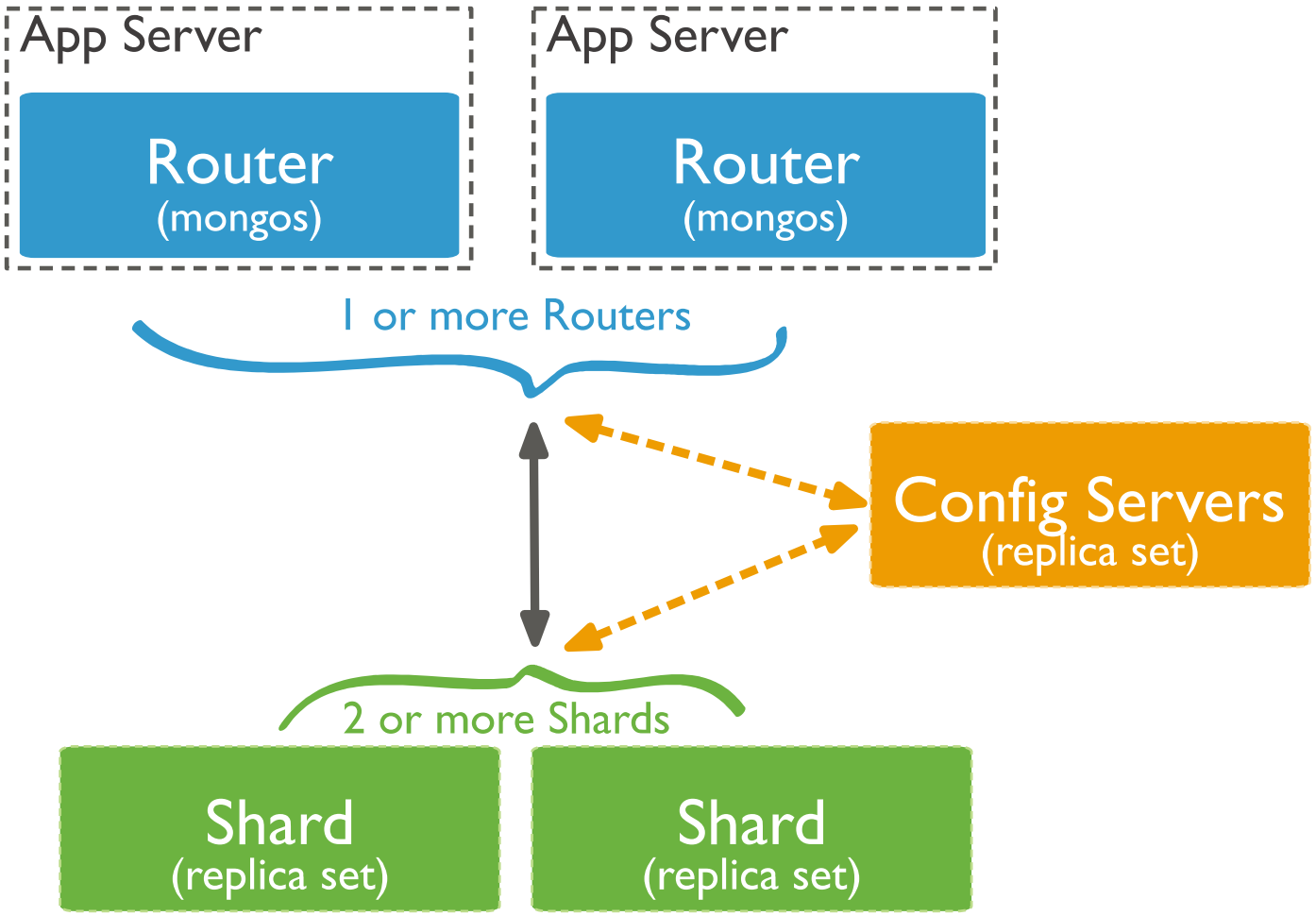
- Sharding is a method for distributing data across multiple machines
- MongoDB uses sharding to support deployments with very large data sets and high throughput operations
- Database systems with large data sets or high throughput applications can challenge the capacity of a single server
- High query rates can exhaust the CPU capacity of the server
- Working set sizes larger than the system's RAM stress the I/O capacity of disk drives
- MongoDB supports horizontal scaling through sharding
- MongoDB shards data at the collection level, distributing the collection data across the shards in the cluster



MongoDB – Sharding

- A MongoDB sharded cluster consists of the following components:
  - shard:
    - Each shard contains a subset of the sharded data
    - Each shard must be deployed as a replica set
  - mongos:
    - The mongos acts as a query router
    - Provides an interface between client applications and the sharded cluster
  - config servers:
    - Config servers store metadata and configuration settings for the cluster
    - As of MongoDB 3.4, config servers must be deployed as a replica set (Configuration Servers Replica Set - CSRS)

MongoDB – Sharding





MongoDB – Sharding

- Shard Keys
  - To distribute the documents in a collection, MongoDB partitions the collection using the shard key
  - The shard key consists of an immutable field or fields that exist in every document in the target collection
  - The shard key is chosen when sharding the collection
  - The choice of shard key cannot be changed after sharding
  - The shard key can be a composite key
  - A sharded collection can have only one shard key



MongoDB – Sharding

- Shard Keys
  - To shard a non-empty collection:
    - The collection must have an index that starts with the shard key
  - To shard empty collections:
    - MongoDB creates the index if the collection does not already have an appropriate index for the specified shard key



MongoDB – Sharding

- Shard Keys
  - For a sharded cluster, the choice of shard key affects the:
    - Performance
    - Efficiency
    - Scalability
  - A cluster with the best possible hardware and infrastructure can be bottlenecked by the choice of shard key
  - The choice of shard key and its backing index can also affect the sharding strategy that your cluster can use



MongoDB – Sharding

- Chunks
  - MongoDB partitions sharded data into chunks
  - Each chunk has an inclusive lower and exclusive upper range based on the shard key
  - MongoDB migrates chunks across the shards in the sharded cluster using the sharded cluster balancer
  - The balancer attempts to achieve an even balance of chunks across all shards in the cluster



MongoDB
Advantages of sharding

- Read/Write:
  - MongoDB distributes the read and write workload across the shards in the sharded cluster
  - Each shard processes a subset of cluster operations
- Storage Capacity:
  - Sharding distributes data across the shards in the cluster
  - Each shard contains a subset of the total cluster data
- High Availability:
  - A sharded cluster can continue to perform partial read / write operations even if one or more shards are unavailable



Overview

- MongoDB
  - Replication
  - Sharding
  - Using MongoDB



Installing MongoDB

- To install MongoDB:
  - Download the package for your OS
  - For Ubuntu 24.04 x64:

1. Get the tgz package from https://www.mongodb.com/try/download/community
2. Unzip: $ tar -zxvf mongodb-linux-x86_64-ubuntu2404-8.0.3.tgz
3. Create a directory for the data: $ mkdir mongo_data
4. Start the server: $ mongod --dbpath mongo_data



Starting a MongoDB instance

- When mongo is started the default host is localhost (127.0.0.1) and the default port is 27017

- If you want to change the host, when you start mongod use --bind_ip and give an IP

```
$ mongod --dbpath mongo_data --bind_ip IP
```

- If there is a binding between the host name and an IP (in /etc/hosts) you can use the hostname instead of the IP to connect to MongoDB

- If you want to change the default port, when you start mongod use --port and give a new port

```
$ mongod --dbpath mongo_data --port 27110
```



Connecting to MongoDB

- To connect to MongoDB use mongosh

```
$ mongosh [--host {IP|hostname}] [--port {port_no}] [DB_name]
```

```
$ mongosh [{IP|hostname}:{port_no}/][DB_name]
```

- If no database name is specified, then mongo will connect directly to the admin database



Replication in MongoDB

- Start multiple instances using mongod with the --replSet ReplicaName parameter

- Local replication example

1. Create a data directory for each node (make sure to have the privileges):

```
$ mkdir -p node1_data node2 _data node3_data
```

2. Start the instances:

```
$ nohup mongod --replSet rs1 --dbpath node1_data --port 27021 > log_n1 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node2_data --port 27022 > log_n2 2>&1 &
$ nohup mongod --replSet rs1 --dbpath node3_data --port 27023 > log_n3 2>&1 &
```



Replication in MongoDB

- Initialize the replication

- Connect to one of the nodes:

```
$ mongosh localhost:27021
```

- Initialize the replica set

```
> rs.initiate( {
   _id: "rs1",
   members: [  { _id : 0, host : "localhost:27021" },  { _id : 1, host
:  "localhost:27022" }, { _id : 2, host : "localhost:27023" }   ]  } )
```

- Verify the status of the replica set

```
> rs.status()
```



Replication in MongoDB

- To read from a secondary:
  - Connect to the secondary node
  - Issue the following command

```
> rs.secondaryOk() // depricated
> db.getMongo().setReadPref("primaryPreferred")
```

  - Disable reading from secondary

```
> db.getMongo().setReadPref("primary")
```



Sharding in MongoDB

- Create the directories for the config and shard nodes

```
$ mkdir -p conf1_data conf2_data conf3_data
$ mkdir -p rs1node1_data rs1node2_data rs1node3_data
$ mkdir -p rs2node1_data rs2node2_data rs2node3_data
$ mkdir -p rs3node1_data rs3node2_data rs3node3_data
```

- Create the configuration servers

```
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf1_data --port 27121 > log_conf1 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf2_data --port 27122 > log_conf2 2>&1 &
$ nohup bin/mongod --configsvr --replSet ConfRepl --dbpath conf3_data --port 27123 > log_conf3 2>&1 &
```



Sharding in MongoDB

- Initialize replication for the config servers

```
$ mongosh localhost:27121
> rs.initiate(  {
   _id: "ConfRepl",
   configsvr: true,
   members: [
                        { _id : 0, host : "localhost:27121" },
                        { _id : 1, host : "localhost:27122" } ,
                        { _id : 2, host : "localhost:27123" }
            ]
} )
```



Sharding in MongoDB

- Start the sharding replica set nodes

```
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node1_data --port 27011  > log_rs1node1 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node2_data --port 27012  > log_rs1node2 2>&1 &
$ nohup bin/mongod --shardsvr --replSet RS1 --dbpath rs1node3_data --port 27013  > log_rs1node3 2>&1 &
```

- And initialize replication

```
$ bin/mongosh localhost:27011
> rs.initiate({
   _id: "RS1",
   members: [
      { _id : 0, host : "localhost:27011" },
      { _id : 1, host : "localhost:27012" },
      { _id : 2, host : "localhost:27013" }
   ]
})
```

Note: Do this 2 more times for the other 2 shards – change the configuration accordingly for each shard



Sharding in MongoDB

- Start a mongos instance

```
$ nohup bin/mongos --configdb
ConfRepl/localhost:27121,localhost:27122,localhost:27123 --port 27130 >
/home/log_mongos 2>&1 &
```

- Connect to the mongos instance and add the shards

```
$ bin/mongosh localhost:27130

> sh.addShard("RS1/localhost:27011")
> sh.addShard("RS1/localhost:27012")
> sh.addShard("RS1/localhost:27013")
> sh.addShard("RS2/localhost:27021")
> sh.addShard("RS2/localhost:27022")
> sh.addShard("RS2/localhost:27023")
> sh.addShard("RS3/localhost:27031")
> sh.addShard("RS3/localhost:27032")
> sh.addShard("RS3/localhost:27033")
```



Sharding in MongoDB

- Verify the sharding status

```
> sh.status()
```

- Create a collection

```
> use BD2
> var vec = [ {x: 0 }]
> vec.length
> for (var i=1; i<100000000; i++){ vec.push({x: i})}
> vec.length
> db.documents.insertMany(vec);
> db.documents.findOne()
```



Sharding in MongoDB

- Enable sharding using on a database **sh.enableSharding("db_name")**

```
sh.enableSharding(“BD2")
```

- Shard the collection using **sh.shardCollection("db_name.collection_name", {key: 1})**

```
sh.shardCollection(“BD2.documents", {_id: 1})
```

- Verify the sharding status

```
sh.status({verbose: true})
```



Sharding in MongoDB
Conclusion

- Start the config servers
- Initialize replication for the config servers
- Create shard replicas
- Initialize replication for each shard replica set
- Start mongos
- Add the replica shards



CRUD Operations in MongoDB

- Insert operation:
  - db.collection_name.insertOne(doc)
  - db.collection_name.insert(doc, {ordered: <boolean>}))
  - db.collection_name.insert([docs])
  - db.collection_name.insertMany([docs], {ordered: <boolean>})



CRUD Operations in MongoDB

- Insert Examples

```
> doc = { name: {firstname: "Ciprian", lastname: "Truica"}, age: 32, courses: [
{name: "BD1", year: 3}, {name: "BD2", year: 4}, {name: "DM", year: 5}],
offices: ["EF208", "PR305"]}

> db.teachers.insert(doc)

> doc = { name: {firstname: "Alexandru", lastname: "Boicea"}, age: 59,
courses: [{ name: "BD1", year: 3}, { name: "BD2", year: 4}, { name: "SCAD",
year: 4}, { name: "ICASE", year: 5}], offices: ["EF309"]}

> db.teachers.insertOne(doc)
```



CRUD Operations in MongoDB

- Insert Examples

```
> docs = [
        { name: {firstname: "Florin", lastname: "Radulescu"}, age: 58,
courses: [ {name: "BD1", year: 3}, {name: "BD2", year: 4}, , {name: "UBD",
year: 4}, {name: "DMDW", year: 5}], offices: ["Vice Dean Office", "EF309"]},
        { name: {firstname: "Sorin", lastname: "Ciolofan"}, age: 37, courses:
[{name: "PAJ", year: 5}, {name: "MN", year: 1}]},
        {name: {firstname: "Dan", lastname: "Schrager"}, courses: [ {name:
"PC", year: 1}, {name: "ABD", year: 5}], offices: ["PR304"]}
]

> db.teachers.insertMany(docs, {ordered: true})
```



CRUD Operations in MongoDB

- Query Documents:
  - db.collection.find(query, projection)
  - db.collection.findOne(query, projection)



CRUD Operations in MongoDB

- Find all:

```
> db.teachers.find()
```

- Find one:

```
> db.teachers.findOne()
```

- Find all under 40 years:

```
> db.teachers.find({age: {$lte: 40}}, {name: 1})
```

- Find all over 35:

```
> db.teachers.find({age: {$gte: 35}}, {name: 1})
```

- Find all with age between 40 and 60 years:

```
> db.teachers.find({age: {$gte: 40, $lte: 60}}, {name: 1})
```



CRUD Operations in MongoDB

- Find all who have the office EF309:

```
> db.teachers.find({offices: "EF309"}, {name: 1})
```

- Find all who do not have an office:

```
> db.teachers.find({offices: {$exists: false}}, {name: 1})
```

- Find all who teach BD1:

```
> db.teachers.find({"courses.name": "BD1"}, {name: 1})
```

- Find all teachers whose courses starting with DM (regex):

```
> db.teachers.find({"courses.name": /^DM/}, {name: 1, "courses.name": 1})
```

- Find all who have courses at the 1 year or his age is over 50:

```
> db.teachers.find({$or: [{"courses.year": 1}, {age: {$gt: 50}}]}, {name: 1})
```



CRUD Operations in MongoDB

- Find all who have courses at the 5 year and his age is under 50:

```
> db.teachers.find({$and: [{"courses.year": 5}, {age: {$lt: 50}}]}, {name: 1})
```

- Find all that have 2 offices:

```
db.teachers.find({offices: {$size: 2}}, {name: 1})
```

- Find all that has as his first office EF309:

```
db.teachers.find({"offices.0": "EF309"}, {name: 1})
```

- Find all that have offices in both EF and PR:

```
db.teachers.find({offices: {$all: [/^EF/, /^PR/]}}, {name: 1})
```



CRUD Operations in MongoDB

- Update:
  - db.collection.update(<filter>, <update>, <options>)
  - db.collection.updateOne(<filter>, <update>, <options>)
  - db.collection.updateMany(<filter>, <update>, <options>)
  - db.collection.replaceOne(<filter>, <replacement>, <options>)



CRUD Operations in MongoDB

- Update courses for teachers who are 32 years old:

```
> db.teachers.updateOne({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}})
```

- Update multiple:

```
> db.teachers.update({age: 32}, {$set: {courses: {name: 'UBD', year: 4}}},
{multi: true})
```

- Update courses for all the teachers over 50 years:

```
> db.teachers.updateMany({age: {$gte: 50}}, {$set: {courses: {name: 'UBD',
year: 4}}})
```

```
> db.teachers.update({age: {$gte: 50}}, {$set: {courses: {name: 'UBD', year:
4}}}, {multi: true})
```



CRUD Operations in MongoDB

- Update a nested document, change the name of course BD1 to UBD for Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "BD1"},
{$set: {"courses.$.name": 'UBD'}})

> db.teachers.update({"name.firstname": "Ciprian", "courses.name": "UBD"},
{$set: {"courses.$.name": 'BD1'}})

> db.teachers.find({"name.firstname": "Ciprian"}, {_id: 0, "courses.name": 1})
```



CRUD Operations in MongoDB

- Add a new course for Dan

```
> db.teachers.update({"name.firstname": "Dan"}, {$push: {courses: {name: 'BD1', year: 3}}})

> db.teachers.find({"name.firstname": "Dan"}, { courses: 1})
```

- Add middle name to Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$set: {"name.middlename": 'Octavian'}})

> db.teachers.find({"name.firstname": "Ciprian"}, { name: 1})
```

- Remove middlename from Ciprian

```
> db.teachers.update({"name.firstname": "Ciprian"}, {$unset: { "name.middlename": ""}})
```



CRUD Operations in MongoDB

- Delete:
  - db.collection.remove(<query>, <justOne>)
  - db.collection.deleteOne(<filter>)
  - db.collection.deleteMany(<filter>)
- Drop a collection
  - db.collection.drop()
- Drop a database
  - db.dropDatabase()



CRUD Operations in MongoDB

- Delete one

```
> db.teachers.remove({"name.firstname": "Ciprian"})

> db.teachers.deleteOne({"name.firstname": "Sorin"})
```

- Delete many

```
> db.teachers.deleteMany({age: {$gte: 50}})

> db.teachers.remove({age: {$gte: 50}}, {justOne: false})
```

- Remove everything

```
> db.teachers.remove({})

> db.teachers.deleteMany({})
```



CRUD Operations in MongoDB

- Drop collection

```
> db.teachers.drop()
```

- Drop database

```
> db.dropDatabase()
```



Indexes in MongoDB

- The _id field acts as a primary key and, when build, an index is automatically created
- Secondary indexes can be added on any field
- Secondary indexes can be:
  - Simple indexes on one field
  - Compound indexes on multiple fields
  - Multikey indexes for array
  - Geospatial indexes for geospatial coordinates
  - Text indexes
  - Hashed indexes to support hash based sharding



Indexes in MongoDB

- Simple indexes on one field:

```
> db.collection.createIndex({"label_1": 1})
```

- Compound indexes on multiple fields

```
> db.collection.createIndex({"label_1": 1, "label_2": -1, ...})
```

- Multikey indexes for array

```
> db.collection.createIndex({"label_1.nested_label_1": 1})
```



Indexes in MongoDB

- Geospatial indexes for geospatial coordinates

```
> db.collection.createIndex({"label": "2d"})
> db.collection.createIndex({"label": "2dsphere"})
```

- Text indexes (one per collection, but can be compound)

```
> db.collection.createIndex({"flabel": "text"})

> db.collection.createIndex({"label_1": "text", "label_2": "text",…})
```

- Hashed indexes to support hash based sharding

```
db.collection.createIndex({"label_1": "hashed"})
```



Indexes in MongoDB

- Simple index

```
> db.documents.createIndex({author: 1})
```

- Compound index

```
> db.documents.createIndex({gender: 1, age: 1})
```

- Multikey index

```
> db.documents.createIndex({"words.word": 1})
```

- Geospatial index

```
> db.documents.createIndex({"geoLocation": "2d"})
```



Indexes in MongoDB

- Text index

```
> db.documents.createIndex({"lemmaText": "text"})
```

- Hashed index

```
> db.documents.createIndex({"date": "hashed"})
```

- Get all indexes for a collection

```
> db.documents.getIndexes()
```

- Drop an Index

```
> db.documents.dropIndex("lemmaText_text")
```

- Drop all indexes except for primary key

```
> db.documents.dropIndexes()
```



Indexes in MongoDB

- Searching with Geospatial indexes
  - To find all the documents near a point (± 1)

```
> db.documents.find( { geoLocation: { $near: [25, 25], $minDistance: 0, $maxDistance: 5 } }, {geoLocation: 1} )
```

  - To find all the documents in the circle with the center [25, 10], and radius 3

```
> db.documents.find( { geoLocation: { $within: { $center: [[25, 10], 3] } } }, {geoLocation: 1} )
```

  - To find all the documents in the rectangle defined by [25, 40], [30, 45]

```
> db.documents.find( { geoLocation: { $within: { $box: [[25, 40], [30, 45]] } } }, { geoLocation: 1 } )
```

  - To find all the documents in the polygon defined by a polygon with 3 points

```
> db.documents.find( { geoLocation: { $within: { $polygon: [[20, 40], [40, 50], [30, 30]] } } },  { geoLocation: 1 } )
```



Indexes in MongoDB

- Searching with Text indexes

```
{
  $text:
    {
      $search: <string>,
      $language: <string>,
      $caseSensitive: <boolean>,
      $diacriticSensitive: <boolean>
    }
}
```



Indexes in MongoDB

- Searching with Text indexes

- Search for a word

```
> db.documents.find( { $text: { $search: "cup", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```

- Search for a multiple word (with an or between them)

```
> db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```



Indexes in MongoDB

- Searching with Text indexes

- Search for a Phrase

```
db.documents.find( { $text: { $search: "\"cup winner\"", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { lemmaText: 1 } )
```

- Search with Text Search Score

```
db.documents.find( { $text: { $search: "cup winner", $language: "english", $caseSensitive: false, $diacriticSensitive: false } } ,  { score: { $meta: "textScore" }, lemmaText: 1, _id: 0})
```



Aggregation in MongoDB

- Aggregation in MongoDB can be done using:
  - Single Purpose Aggregation methods
  - Aggregation Pipeline
  - Map-Reduce



Single purpose aggregation methods

- Count: db.collection.count(query)

```
> db.documents.countDocuments({lemmaTextLength: 10})
```

- Distinct: db.collection.distinct(field, query)

```
> db.documents.distinct("age")

> db.documents.distinct("age", {gender: "female"}).sort()

> db.documents.distinct("age", {gender: "female"}).reverse()
```



Aggregation Pipeline (I)

- MongoDB's aggregation framework is modeled on the concept of data processing pipelines
- Documents enter a multi-stage pipeline that transforms the documents into an aggregated result
- The most basic pipeline stages provide filters that operate like queries and document transformations that modify the form of the output document
- The operation's order matters



Aggregation Pipeline (I)

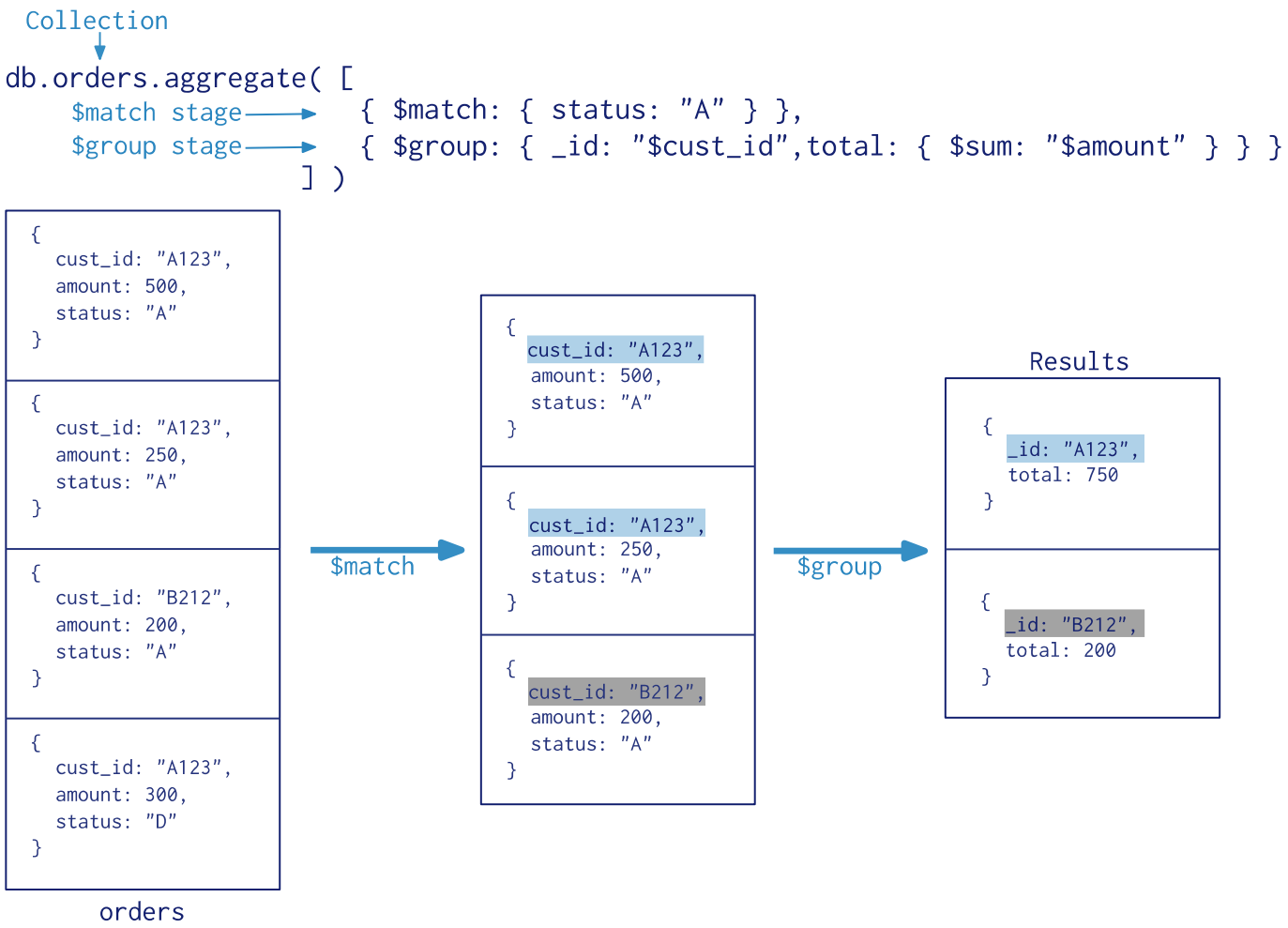
- Other pipeline operations provide tools for grouping and sorting documents by specific field or fields as well as tools for aggregating the contents of arrays, including arrays of documents
- In addition, pipeline stages can use operators for tasks such as calculating the average or concatenating a string
- The pipeline provides efficient data aggregation using native operations within MongoDB
- It is the preferred method for data aggregation in MongoDB
- The aggregation pipeline can operate on a sharded collection



Aggregation Pipeline

- The aggregation pipeline can use indexes to improve its performance during some of its stages
- Aggregation pipeline operations have an optimization phase which attempts to reshape the pipeline for improved performance
- The aggregation pipeline can determine if it requires only a subset of the fields in the documents to obtain the results
- If so, the pipeline will only use those required fields, reducing the amount of data passing through the pipeline

Aggregation Pipeline





Aggregation Pipeline

- Aggregation Pipeline (minimal) syntax – the operations' order matters

```
db.collection.aggregate(
   { $match: <query> },
   { $unwind: <array> },
   { $project: <projection>},
   { $group: <aggregation_group, aggregation_functions> },
   { $sort: <sorting_fields> },
   { $limit: <number>}
)
```



Aggregation Pipeline

- Extract Top-K Keywords

```
TFIDF_PA_TopKWords = function(q, k){
    var docs = Array();
    noDocs = db.documents.count(q);
    // aggregate function that computes the number of times the word appears in the corpus and the TF sum
    db.documents.aggregate([
      { $match: q },
      { $unwind: "$words" },
      { $project: { word: '$words.word', wtf: '$words.tf' } },
      { $group: { _id: "$word", sum_tf: { $sum: "$wtf" }, count: { $sum: 1 } } }
    ]).forEach(function(elem){
      var tfidf = elem.sum_tf * (1 + Math.log(noDocs/elem.count));
      var doc = {word: elem._id, TFIDF: tfidf};
      docs.push(doc);
    });
    db.vocabulary.insert(docs);
    var items = db.vocabulary.find().sort({TFIDF: -1}).limit(k);
    return items;
  }
```



Map Reduce

- MongoDB also provides map-reduce operations to perform aggregation (deprecated)

- In general, map-reduce operations have two phases:
  - A map stage that processes each document and emits one or more objects for each input document
  - A reduce phase that combines the output of the map operation

- Optionally, map-reduce can have a finalize stage to make final modifications to the result

- Like other aggregation operations, map-reduce can specify a query condition to select the input documents as well as sort and limit the results.



Map Reduce

- Map-reduce uses custom JavaScript functions to perform the
  - Map operation
  - Reduce operations
  - As well as the optional finalize operation

- While the custom JavaScript provide great flexibility compared to the aggregation pipeline, in general, map-reduce is less efficient and more complex than the aggregation pipeline

- Map-reduce can operate on a sharded collection

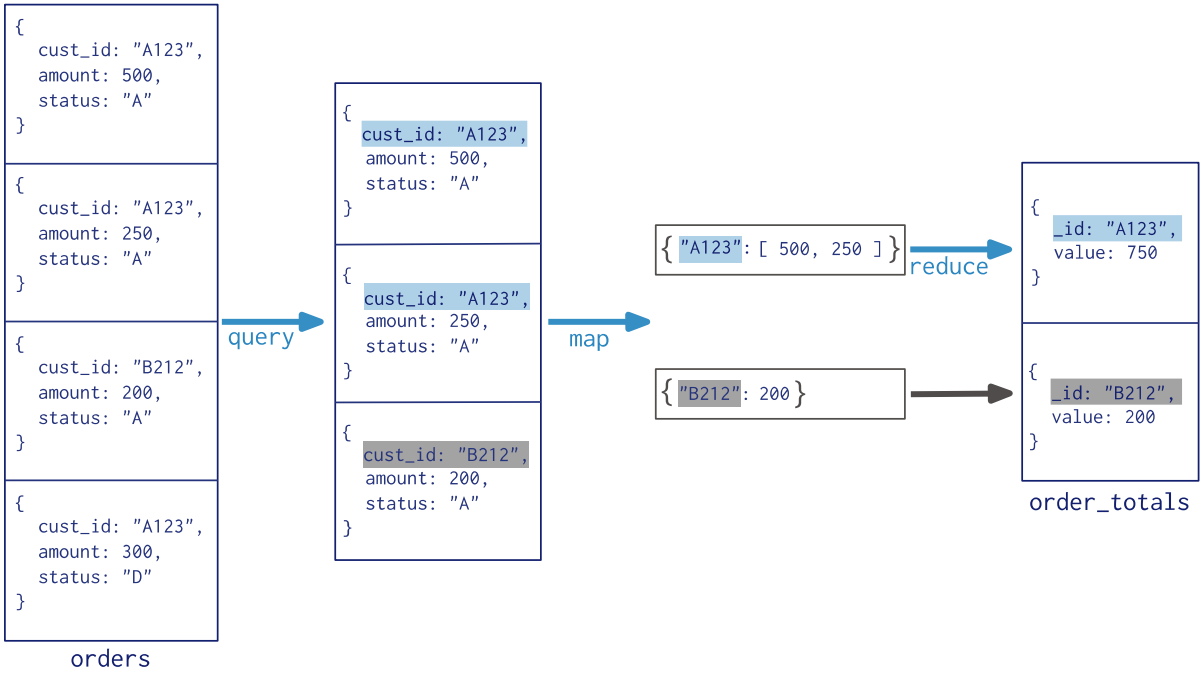
- Map-reduce operations can also output to a sharded collection



Map Reduce

Collection

```
db.orders.mapReduce(
        map      →  function() { emit( this.cust_id, this.amount ); },
        reduce   →  function(key, values) { return Array.sum( values ) },
                    {
        query    →     query: { status: "A" },
        output   →     out: "order_totals"
                    }
                )
```





Map Reduce

- Map Reduce (minimal) syntax

```
db.collection.mapReduce(
   <mapFunction>,
   <reduceFunction>,
   {
      query: <filter_query>,
      out: <output_collection>,
      scope: <external variable>,
      finalize: <finalize_function,
      sort: <document>,
      limit: <number>,
   }
);
```



Map Reduce

- Extract Top-K Keywords

```
TFIDF_MR_TopKWords = function(q, k){
    // compute the number of times the word apear in the corpus and the TF sum
    mapFunction = function() {
            for (var idx=0; idx<this.words.length; idx++){
              emit(this.words[idx].word, { count: 1, stf: this.words[idx].tf });
            } }
    reduceFunction = function(key, values) {
              reducedVal = { count: 0, stf: 0 };
              for (var idx = 0; idx < values.length; idx++) {
                reducedVal.count += values[idx].count;
                reducedVal.stf += values[idx].stf;
              }
              return reducedVal;};
    // The function  that computes the TFIDF
    finalizeFunction = function(key, reducedVal){
      reducedVal.tfidf = reducedVal.stf * (1 + Math.log(noDocs/reducedVal.count));
      return reducedVal; }
    var noDocs = db.documents.count(q);
    db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "vocabulary", scope: {noDocs: noDocs}, finalize:
finalizeFunction});
    // print the values
    var items = db.vocabulary.find().sort({"value.tfidf": -1}).limit(k).addOption(DBQuery.Option.noTimeout);
    return items; }
```

Bibliography

- MongoDB Documentation: https://www.mongodb.com/docs/